

Guia de Desarrollo de Software Corporativo

Version 3.1 - Julio 2026

Departamento de Ingenieria de Software

TechCorp S.A. de C.V.

*Este documento establece las guias, estandares y mejores practicas
para el desarrollo de software dentro de la organizacion.*

Confidencial - Uso exclusivo del personal autorizado.

1. Introduccion

Esta guia tiene como objetivo establecer los estandares y mejores practicas para el desarrollo de software en TechCorp S.A. de C.V.

El documento esta dirigido a todos los desarrolladores, lideres de equipo y responsables de calidad que participen en proyectos de desarrollo.

El cumplimiento de estas guias es obligatorio para todos los proyectos internos. Las excepciones deben ser aprobadas por la direccion de ingenieria.

Esta guia se actualiza trimestralmente conforme evolucionan las tecnologias y las necesidades del negocio.

2. Estándares de Código

Todos los proyectos deben seguir los estándares de codificación establecidos para garantizar consistencia, legibilidad y mantenibilidad del código fuente.

Convenciones de Nomenclatura

- Las clases deben usar PascalCase: UserProfile, OrderService, NotificationHandler.
- Los métodos y propiedades deben usar camelCase: getUserById, calculateTotal, isValid.
- Las constantes deben usar UPPER_SNAKE_CASE: MAX_RETRY_COUNT, DEFAULT_TIMEOUT.
- Los campos privados deben usar _camelCase: _databaseConnection, _logger.
- Las interfaces deben comenzar con I: IUserRepository, IEmailService.

Estructura de Archivos

- Cada archivo debe contener una sola clase pública.
- Los archivos de servicios deben nombrarse con sufijo Service: UserService.cs, PaymentService.cs.
- Los archivos de repositorio deben nombrarse con sufijo Repository: UserRepository.cs.
- Los archivos de controlador deben nombrarse con sufijo Controller: AuthController.cs.

Reglas de Formato

- Indentación de 4 espacios (no tabuladores).
- Longitud máxima de línea: 120 caracteres.
- Los archivos deben terminar con una línea en blanco.
- Using statements ordenados: System primero, luego librerías externas, luego namespaces del proyecto.

3. Control de Versiones con Git

El control de versiones es fundamental para cualquier proyecto de software. En TechCorp utilizamos Git como sistema de control de versiones distribuido con GitHub como plataforma de alojamiento.

Estrategia de Ramas

- main: Rama principal, siempre desplegable en produccion.
- develop: Rama de integracion donde se combinan todas las funcionalidades.
- feature/nombre-descripcion: Ramas para nuevas funcionalidades (ej: feature/user-authentication).
- hotfix/nombre-descripcion: Correcciones urgentes para produccion.
- release/version: Preparacion de nuevas versiones para despliegue.

Reglas de Commits

- Cada commit debe ser atomico y representar un cambio logico completo.
- Los mensajes deben seguir el formato Conventional Commits: tipo(scope): descripcion.
- Tipos permitidos: feat, fix, docs, style, refactor, test, chore.
- Ejemplos: feat(auth): add JWT token validation, fix(api): handle null response in user endpoint.

Pull Requests

- Todo cambio debe ser revisado por al menos un desarrollador senior antes de merge.
- Los PRs deben incluir descripcion clara, tickets de referencia y evidencia de pruebas.
- La rama develop debe mantenerse siempre estable, sin errores de compilacion.

4. Revision deCodigo

Las revisiones de codigo son un proceso obligatorio que garantiza la calidad, seguridad y consistencia del codigo antes de ser integrado a la rama principal.

Proceso de Revision

- El autor del codigo crea un Pull Request con la descripcion del cambio.
- Se asignan al menos dos revisores: uno del mismo equipo y uno de otro equipo.
- Los revisores tienen 48 horas para completar la revision.
- Si se solicitan cambios, el autor tiene 24 horas para realizar las correcciones.

Criterios de Aprobacion

- El codigo cumple con los estandares de nomenclatura y formato.
- No existen vulnerabilidades de seguridad conocidas (injection, XSS, etc.).
- Las pruebas unitarias cubren al menos el 80% del codigo nuevo.
- La documentacion esta actualizada si aplica.
- No hay duplicacion significativa de codigo con el que ya existe.
- El rendimiento no se ve afectado de manera significativa.

Herramientas de Soporte

- SonarQube para analisis estatico y deteccion de deuda tecnica.
- ReSharper para inspeccion de codigo en C#.
- GitHub Actions para ejecucion automatica de pruebas en cada PR.
- Dependabot para mantenimiento automatico de dependencias.

5. Pruebas de Software

Las pruebas son un pilar fundamental del desarrollo de software de calidad. Todo código producido debe ser probado antes de llegar a producción.

Niveles de Prueba

- Unitarias: Validan métodos y funciones individuales. Objetivo: 80% de cobertura.
- Integración: Validan la interacción entre componentes o servicios.
- End-to-End: Simulan flujos completos del usuario desde la interfaz.
- Regresión: Se ejecutan automáticamente antes de cada despliegue.
- Carga: Miden el comportamiento del sistema bajo alta demanda.

Frameworks Recomendados

- .NET: xUnit con Moq para mocks, FluentAssertions para aserciones.
- JavaScript: Jest para unitarias, Cypress para E2E.
- Python: pytest con pytest-cov para cobertura.

Datos de Prueba

- Nunca usar datos reales de producción en entornos de prueba.
- Utilizar fábricas de datos (AutoFixture, Bogus) para generar datos automáticos.
- Los entornos de prueba deben ser idempotentes: cada ejecución parte del mismo estado inicial.

6. Seguridad en Desarrollo

La seguridad debe ser parte integral del ciclo de desarrollo. No es un paso final sino una practica continua desde el diseno hasta el despliegue.

Principios de Seguridad

- No confiar en la entrada del usuario: siempre validar y sanitizar datos de entrada.
- Principio de minimo privilegio: cada componente solo debe tener los permisos necesarios.
- Defensa en profundidad: implementar multiples capas de proteccion.
- Seguridad por diseno: considerar amenazas desde la fase de diseno.

Practicas Obligatorias

- Nunca almacenar contraseñas en texto plano. Usar bcrypt o Argon2.
- Utilizar parametrización en consultas SQL para prevenir inyección SQL.
- Implementar HTTPS en todos los endpoints de la API.
- Usar CORS configurado restrictivamente, no permitir orígenes wildcard.
- Las llaves secretas y tokens deben almacenarse en variables de entorno o gestores de secretos como Azure Key Vault.
- Realizar análisis de dependencias para detectar vulnerabilidades conocidas (OWASP Dependency Check).

Revision de Seguridad

- Cada sprint incluye una revision de seguridad del código nuevo.
- Se realizan pruebas de penetración trimestrales por el equipo de seguridad.
- Los hallazgos se documentan en Jira con prioridad asignada según CVSS.

7. Despliegue y CI/CD

La integracion continua y el despliegue continuo son practicas esenciales para mantener la agilidad y confiabilidad en la entrega de software.

Pipeline de CI/CD

- Fase 1: Compilacion automatica al detectar cambios en la rama develop.
- Fase 2: Ejecucion de pruebas unitarias y de integracion.
- Fase 3: Analisis estatico con SonarQube y verificacion de cobertura minima.
- Fase 4: Build del artefacto Docker y push al registro de containers.
- Fase 5: Despliegue automatico a staging para validacion manual.
- Fase 6: Promocion a produccion con aprobacion del tech lead.

Entornos

- Desarrollo: Maquina local del desarrollador con datos simulados.
- Staging: Replica exacta de produccion con datos de prueba anonimizados.
- Produccion: Entorno final con datos reales y alta disponibilidad.

Rollback

- Todo despliegue debe poder revertirse en menos de 5 minutos.
- Se mantienen las ultimas 3 versiones de cada servicio como respaldo.
- En caso de incidente critico, el tech lead puede activar rollback inmediato sin aprobacion adicional.

8. Documentacion Tecnica

La documentacion tecnica es tan importante como el codigo mismo. Un sistema sin documentacion adecuada se convierte en deuda tecnica critica.

Tipos de Documentacion

- README: Descripcion del proyecto, instrucciones de instalacion y uso basico.
- API Docs: Generadas automaticamente con Swagger/OpenAPI para todos los endpoints.
- ArchiTecture.md: Diagramas de arquitectura, decisiones de diseno y dependencias.
- CHANGELOG: Registro cronologico de cambios relevantes en cada version.
- README de cada microservicio: Descripcion, endpoints, variables de entorno y como ejecutar localmente.

Estandares de Documentacion

- Toda API REST debe tener documentacion Swagger actualizada.
- Los diagramas de arquitectura deben mantenerse en formato Mermaid o PlantUML.
- La documentacion se almacena en el repositorio del proyecto en una carpeta docs/.
- Los diagramas de secuencia deben cubrir los flujos criticos del negocio.

Responsabilidades

- Cada desarrollador es responsable de documentar el codigo que escribe.
- Los tech leads revisan la documentacion como parte del proceso de merge.
- El equipo de arquitectura mantiene los diagramas de alto nivel actualizados trimestralmente.

9. Gestion de Dependencias

Las dependencias de terceros son una parte inevitable del desarrollo moderno, pero deben ser gestionadas cuidadosamente para evitar vulnerabilidades y problemas de compatibilidad.

Politica de Dependencias

- Solo se permiten dependencias aprobadas por el comite tecnico.
- Todas las dependencias deben tener licencia compatible (MIT, Apache 2.0, BSD).
- Se prefiere usar la menor cantidad de dependencias posible (principio de minimalismo).
- Cada nueva dependencia debe ser justificada y documentada en el ADR (Architecture Decision Record).

Actualizacion

- Dependabot crea PRs automaticos para actualizaciones de seguridad cada semana.
- Las actualizaciones menores se revisan y mergean dentro de 5 dias habiles.
- Las actualizaciones mayores requieren pruebas de regresion completas antes de merge.
- Se realiza un audit de dependencias el primer lunes de cada mes.

Herramientas

- NuGet para proyectos .NET con audit habilitado.
- npm audit y Snyk para proyectos JavaScript/TypeScript.
- pip-audit para proyectos Python.
- OWASP Dependency-Check como herramienta unificada de escaneo.

10. Metodologia Agil - Scrum

TechCorp utiliza Scrum como metodologia de desarrollo iterativo e incremental. Todos los equipos de desarrollo deben seguir el proceso definido.

Ceremonias Obligatorias

- Sprint Planning (inicio del sprint): Definicion del objetivo y seleccion de historias de usuario.
- Daily Standup (diario): Sincronizacion de 15 minutos por equipo.
- Sprint Review (fin del sprint): Demostracion del incremento a stakeholders.
- Sprint Retrospective: Evaluacion del proceso y definicion de mejoras.

Duracion de Sprints

- Cada sprint tiene una duracion fija de 2 semanas.
- Los sprints comienzan los lunes y terminan los viernes.
- No se deben extender sprints ni fusionar historias de usuario entre sprints.

Roles

- Product Owner: Prioriza el backlog y representa las necesidades del negocio.
- Scrum Master: Facilita el proceso y elimina impedimentos.
- Development Team: Equipo auto-organizado de 5 a 9 personas.

Metricas

- Velocidad del equipo: Historias de usuario completadas por sprint.
- Lead Time: Tiempo desde que se crea una historia hasta que se despliega.
- Bug Escape Rate: Numero de defectos que llegan a produccion.
- Sprint Goal Achievement: Porcentaje de sprints donde se alcanza el objetivo.

11. Preguntas Frecuentes

P: Que pasa si un desarrollador no esta de acuerdo con una decision tecnica?

R: Debe expresar su opinion durante la revision de diseno. Si despues de la discusion la decision se mantiene, debe acatarla y documentar las razones en el ADR correspondiente.

P: Puedo usar un framework que no esta en la lista de aprobados?

R: No directamente. Debes presentar una propuesta formal al comite tecnico con justificacion tecnica, benchmarks y analisis de riesgos. El comite evaluara en su proxima reunion mensual.

P: Cual es el tiempo maximo permitido para resolver un bug critico en produccion?

R: Los bugs criticos (servicio completamente caido o fuga de datos) deben resolverse en un plazo maximo de 4 horas. Los bugs altos tienen un SLA de 24 horas.

P: Se permite trabajar de forma remota?

R: Si, el trabajo remoto esta permitido hasta 3 dias por semana. Los martes y jueves son presenciales obligatorios para reuniones de equipo. Los desarrolladores senior pueden negociar condiciones adicionales con su manager.

P: Que herramientas de comunicacion se usan internamente?

R: Slack para comunicacion diaria, Microsoft Teams para reuniones, Jira para gestion de proyectos, Confluence para documentacion y GitHub para codigo fuente.

P: Como se maneja la capacitacion continua del equipo?

R: Cada desarrollador tiene un presupuesto anual de 500 USD para cursos, certificaciones y conferencias. Ademias, se realizan tech talks internas cada viernes por la tarde donde un miembro del equipo presenta un tema tecnico.

12. Informacion de Contacto

Para consultas, sugerencias o reportes relacionados con esta guía de desarrollo, contactar al equipo de Ingeniería de Software:

Correo electronico: ingenieria@techcorp.com

Canal de Slack: [#dev-guidelines](#)

Telefono interno: ext. 7890

Horario de atencion: Lunes a Viernes de 9:00 a 18:00

Para emergencias tecnicas fuera de horario: +52 55 9876 5432

Director de Ingenieria: Arq. Roberto Martinez

Correo directo: roberto.martinez@techcorp.com